YOLOv8-Seg Pothole Model Pipeline

From Raw Dataset to Deployed Area-Calculation Microservice

1 — Data Ingestion & Curation

- ▸ Source dataset: Road Hazards Dataset (road-infrastructure defects)
- ▸ Filter for target classes: potholes, longitudinal cracks

Output: Raw labeled image pool

2 — Data Annotation (Segmentation)

- ▸ Annotate using CVAT / Roboflow
- ▸ Draw precise polygon masks (not bounding boxes) to enable area math

Output: Pixel-accurate mask annotations

3 — Preprocessing & Augmentation

- ▸ Resize images to standard tensor size (e.g., 640×640)
- ▸ Augment: brightness shifts, rotations, noise (simulates poor citizen-camera quality)

Output: Training-ready tensor batches

4 — Model Training (PyTorch)

- ▸ Base model: YOLOv8-Seg, pretrained on COCO
- ▸ Fine-tune weights on the custom road-defect dataset
- ▸ Optimize for mask precision (mAP@0.5:0.95)

Output: Fine-tuned .pt model weights

Trained on Google Colab

External GPU (outside IDE)

CPU too slow for training — GPU used

5 — Model Export & Optimization

- ▸ Export .pt weights → ONNX / TensorRT for faster server-side inference
- ▸ Download weights from Colab and integrate into the local FastAPI codebase

Output: Optimized inference-ready model, ready for local deployment

6 — Deployment & Serving: Python FastAPI Microservice

Custom Post-Processing Script

a — Extract mask pixel count

Sum segmented pixels belonging to the defect mask

b — Detect reference marker

Locate a known-size object in frame (e.g., ruler or coin)

c — Real-world area conversion

`(Defect Pixels / Marker Pixels) × Known Marker Area`

= Defect Area (cm²)

Runs synchronously inside the FastAPI response cycle before returning to the client

Served Output

`{ boundingBox, maskArea, defectAreaCm2 }`

Pipeline Phase Legend

- Data phase — sourcing, labeling, and preparing training data
- Training phase — fine-tuning YOLOv8-Seg on custom defect data
- Optimization phase — export for fast, production-grade inference
- Serving phase — FastAPI microservice + geometric post-processing math

This is the internal engine behind "Microservice B" in the fusion architecture diagram